

Project name:
Design a calculator

NAME OF THE STUDENT: Shyantana Kundu

DEPARTMENT: CSE

SECTION: C

CLASS ROLL NO: 190

ENROLLMENT NO: 12022002003028

SUBJECT - IT WORKSHOP LAB

SUBJECT CODE - PCCCS392



INTRODUCTION

"A Calculating Calculator ;)" - A Calculator

Overview:

In the realm of computational tools, "A Calculating Calculator ;)" stands as a testament to meticulous design and robust functionality. This MATLAB project is a versatile calculator that combines intricate coding with failproof input mechanisms, offering users a reliable and user-friendly mathematical companion.

I. Intricate Coding and Failproof Inputs:

The calculator's foundation lies in intricate coding, ensuring precision and reliability. It boasts failproof input mechanisms, meticulously validating user inputs at every stage to guarantee accurate calculations. This emphasis on error prevention enhances the overall user experience.

II. Feature-Rich Design:

This project goes beyond conventional calculators, offering a plethora of mathematical functions accessible through an intuitive user interface. The design strikes a balance between functionality and aesthetics, providing users with a wholesome and visually appealing tool.

III. Versatility in Mathematical Operations:

From basic arithmetic operations like addition and subtraction to advanced functions such as exponentiation, logarithms, factorials, and permutations, "A Calculating Calculator ;)" is a comprehensive solution for diverse mathematical computations. Its ability to handle complex mathematical tasks sets it apart in the domain of calculator applications.

IV. Compact and User-Centric:

The calculator's compact design doesn't compromise on features. It ensures ease of use with a generic yet intuitive user interface. This user-centric approach is aimed at making mathematical calculations accessible to users of all levels, whether they are students, professionals, or enthusiasts.

ALGORITHM/PSEUDOCODE OF IMPLEMENTATION

1. BEGIN:
2. Initialize `c`, `n1`, `n2`, `n3`, `r`, `s`, `f`, `op`.
3. Set up user interface components.
4. Calculator Initialization:
 - Observe for a change in the 'EnterNumberEditField' value.
 - When the 'EnterNumberEditField' value changes:
 - `c` <- EnterNumberEditField.Value.
 - If `c` is empty or not numeric, display an error message, reset the edit field, and disable buttons.
 - Else, set `n1` <- EnterNumberEditField.Value.
 - If `f` is 0 or `op` is 0:
 - Display the result as " = n1".
 - Enable buttons.
 - Set `f` <- 1.
 - Disable 'Button'.
 - Else:
 - If `op` is 9, 10, or 12, round `n1`.
 - Display the result as `s + n1`.
 - Enable 'Button'.

5. Performing Calculations:
 - Observe button presses for various mathematical operations.
 - When a button is pressed, execute the corresponding mathematical operation:

```
function mathsop(app)
```

```
% Check the value of the 'op' variable and perform the corresponding operation
```

```
switch app.op
```

```
case 1
```

```
app.r = app.n3 + app.n1;
app.s = app.n3 + " + " + app.n1 + " = " + app.r;
app.n3 = app.r;
```

```
case 2
```

```
app.r = app.n3 - app.n1;
app.s = app.n3 + " - " + app.n1 + " = " + app.r;
app.n3 = app.r;
```

```
case 3
```

```
app.r = app.n3 * app.n1;
app.s = app.n3 + " * " + app.n1 + " = " + app.r;
app.n3 = app.r;
```

```
case 4
```

```
app.r = app.n3 / app.n1;
app.s = app.n3 + " / " + app.n1 + " = " + round(app.r, 2);
app.n3 = app.r;
```

```
case 5
```

```
app.r = app.n3 ^ app.n1;
app.s = app.n3 + "^ ( " + app.n1 + " ) = " + round(app.r, 2);
app.n3 = app.r;
```

```
case 6
```

```
app.r = app.n3 ^ (1 / app.n1);
```

```

    app.s = "(" + app.n3 + ")^( 1/" + app.n1 + " ) = " + round(app.r, 2);
    app.n3 = app.r;
case 7
    app.r = log10(app.n3);
    app.s = "log_10( " + app.n3 + " ) = " + round(app.r, 2);
    app.n3 = app.r;
case 8
    app.r = 10 ^ app.n3;
    app.s = "10^( " + app.n3 + " ) = " + round(app.r, 2);
    app.n3 = app.r;
case 9
    app.r = factorial(round(app.n3)) / factorial(round(app.n3 - app.n1));
    app.s = app.n3 + "P" + app.n1 + " = " + round(app.r, 2);
    app.n3 = app.r;
case 10
    app.r = factorial(round(app.n3)) / (factorial(round(app.n3 - app.n1)) * factorial(round(app.n1)));
    app.s = round(app.n3) + "C" + round(app.n1) + " = " + round(app.r, 2);
    app.n3 = app.r;
case 11
    app.r = 1 / app.n3;
    app.s = "1/( " + app.n3 + " ) = " + app.r;
case 12
    app.r = factorial(round(app.n3));
    app.s = "(" + round(app.n3) + " )! = " + round(app.r, 2);
end
% Disable 'EnterNumberEditField'
% app.EnterNumberEditField.Editable = "off";
end
- Update the result `r` based on the operation.
- Update the display string `s`.
- Enable/Disable buttons accordingly.
- When 'Button' is pressed, call `mathsop`.
- Set `n3` <- `r`.
- Set `f` <- 2.
- Display the result.
- Enable 'Button', 'EnterNumberEditField', 'CLRButton'.

```

6. Error Handling:

- If 'CLRButton' is pressed:
- Reset the display and variables.
- Disable buttons.

7. Component Initialization:

- Create UIFigure and components.
- Set UIFigure properties.

8. Callbacks that Handle Component Events:

- Define functions to handle events such as button presses and value changes.
- Implement input validation, error handling, and display updates.

9. END

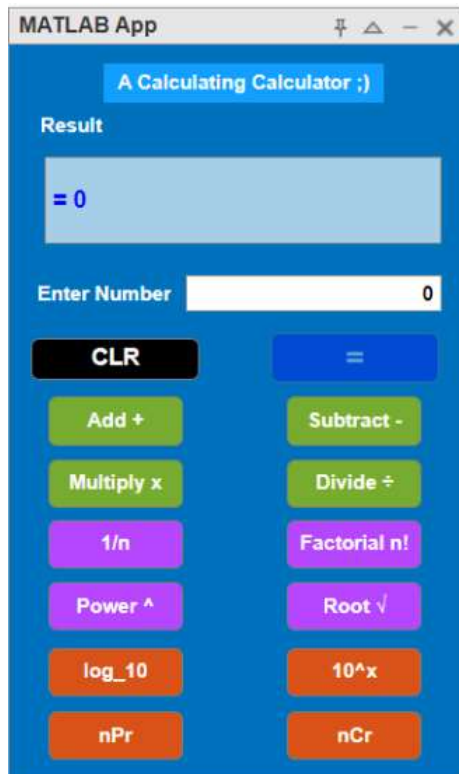
RESULT

Fig 1. GUI of the Calculator after first Start-up

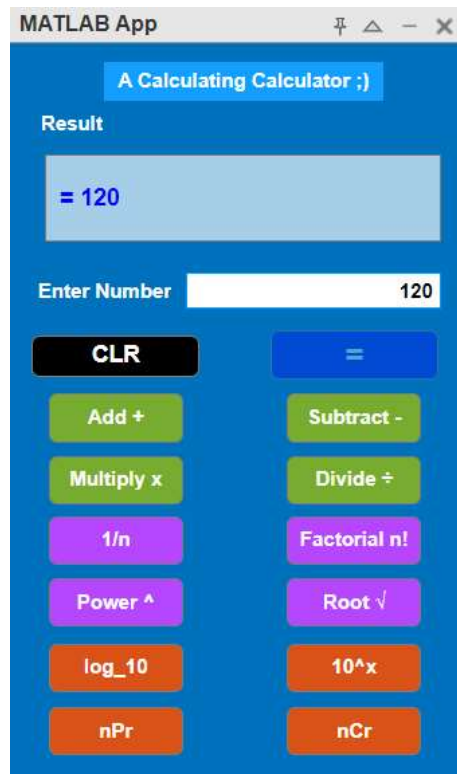


Fig 2. GUI of the Calculator after entering certain number and pressing 'Enter'

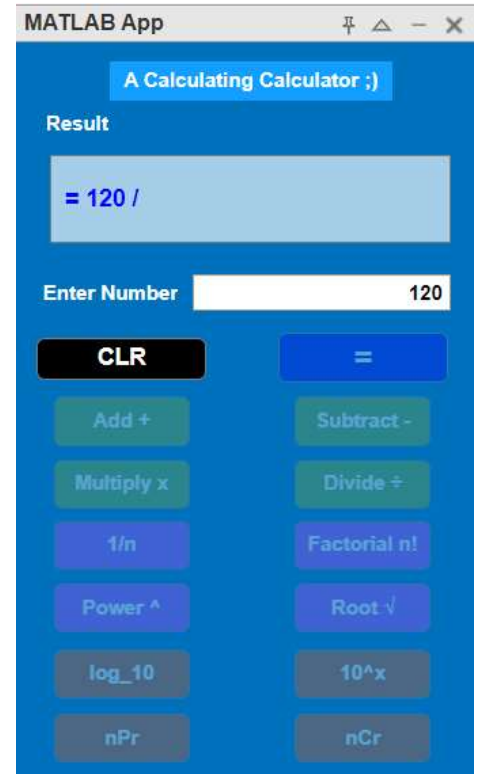


Fig 3.1 Now after pressing Divide button result field shows entered number preceded by = sign and followed by the operator sign



Fig 3.2 Entering the Divisor and pressing Enter

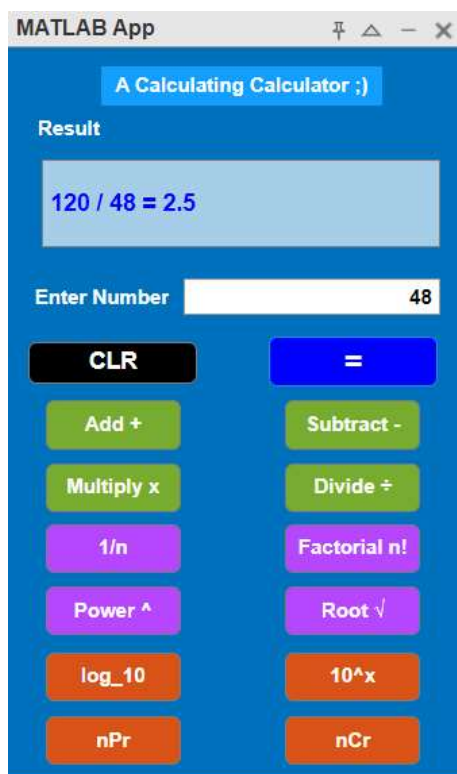


Fig 3.3 After pressing = button

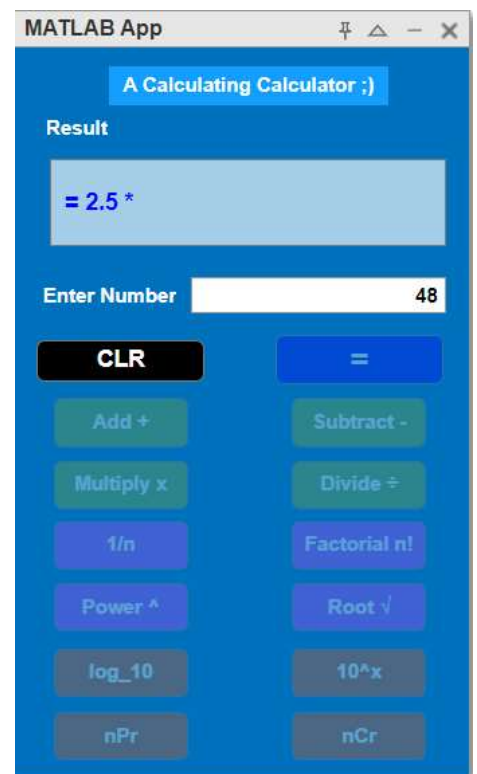


Fig 3.4 The result is stored in memory and then another operator * is pressed

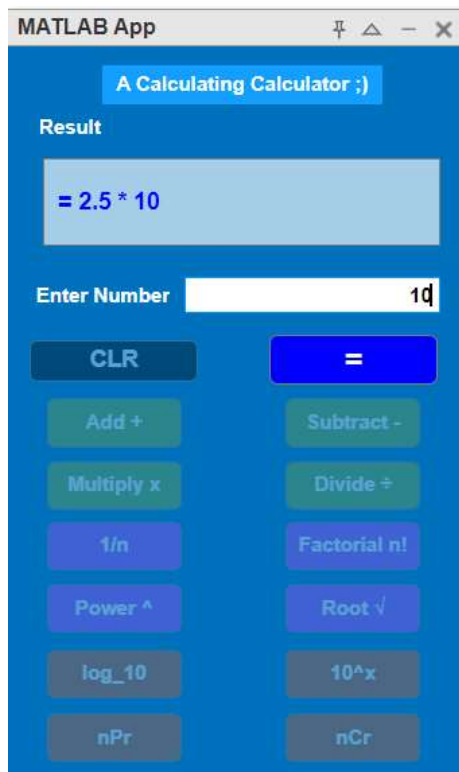


Fig 3.5 Entering the multiplier

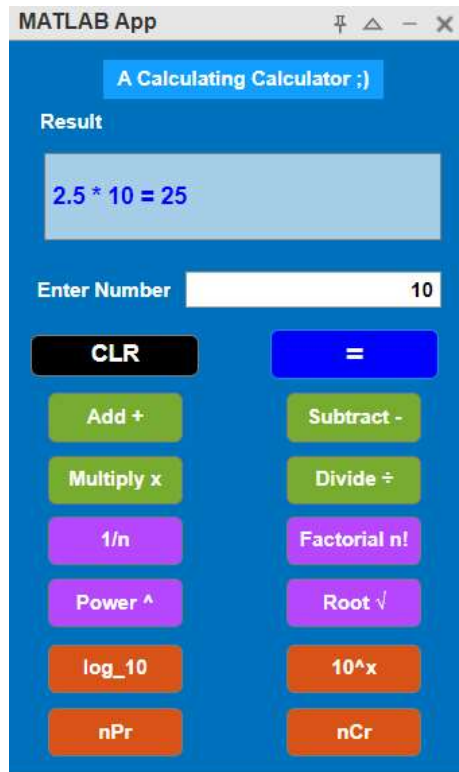


Fig 3.6 Pressing = button to display the result

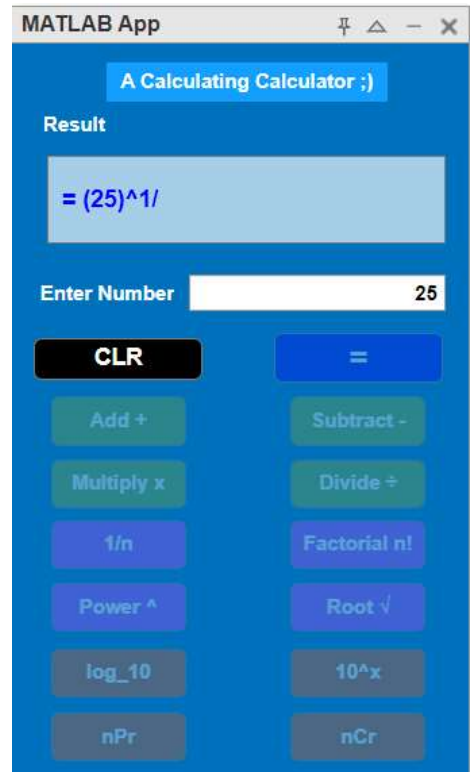


Fig 3.7 Pressing root button to gain root of the result stored in memory

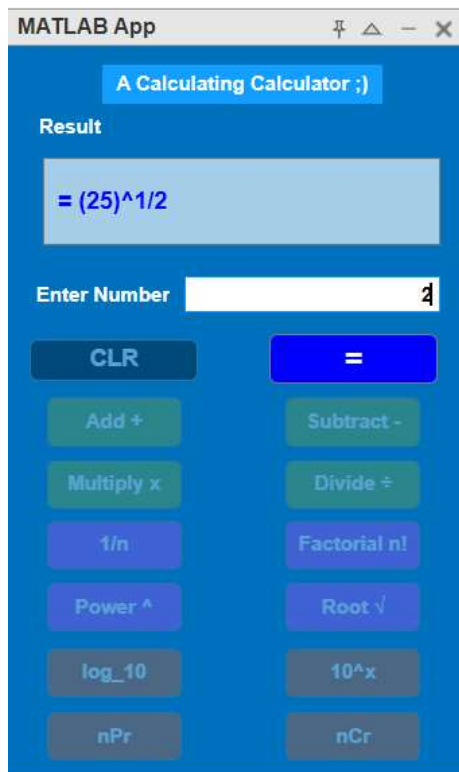


Fig 3.8 Entering the number as per desired root to be obtained (here, sq. root)

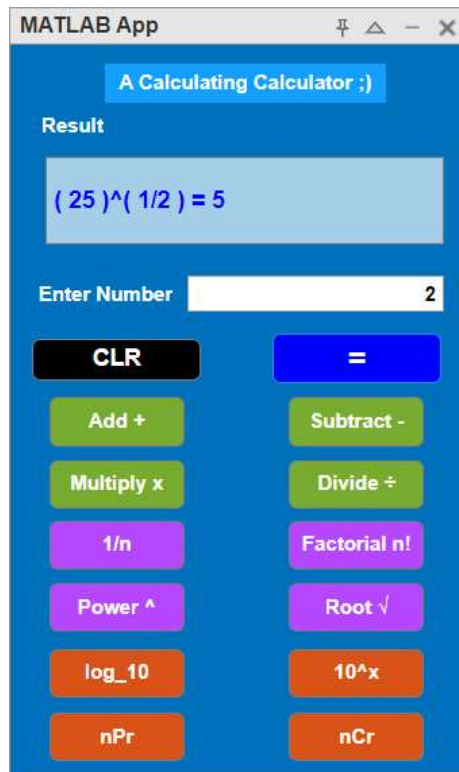


Fig 3.9 Pressing = button to display the result

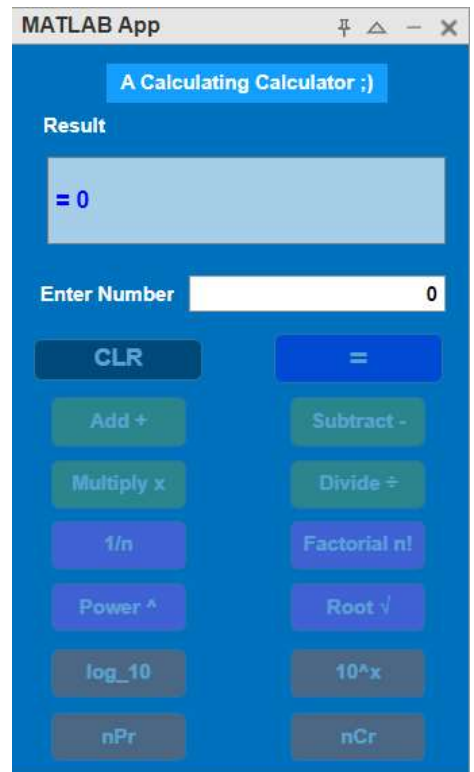


Fig 3.10 pressing CLR button to clear all memory of the calculator and reset it afresh



Fig 4.1 Reciprocal function

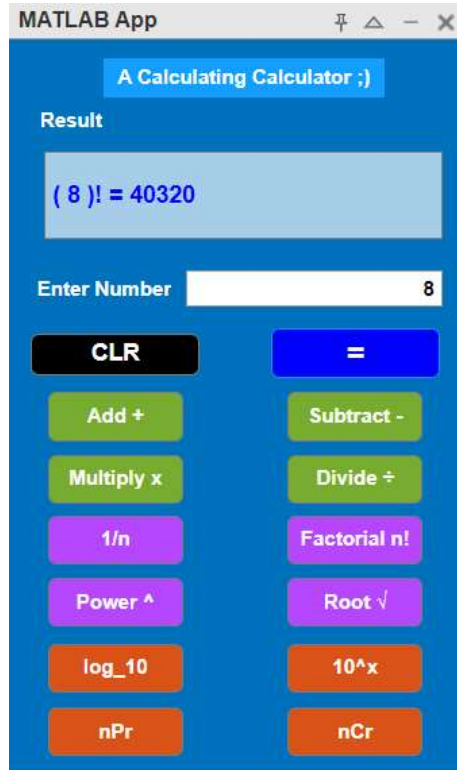


Fig 4.2 Factorial function



Fig 4.3 Power function

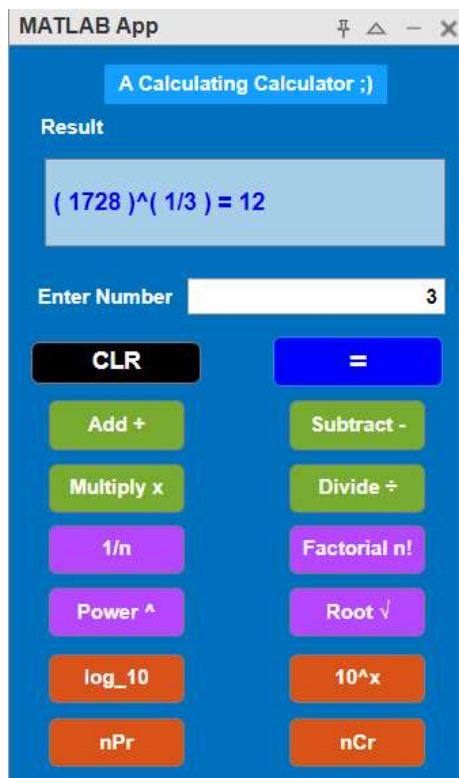


Fig 4.4 Root function

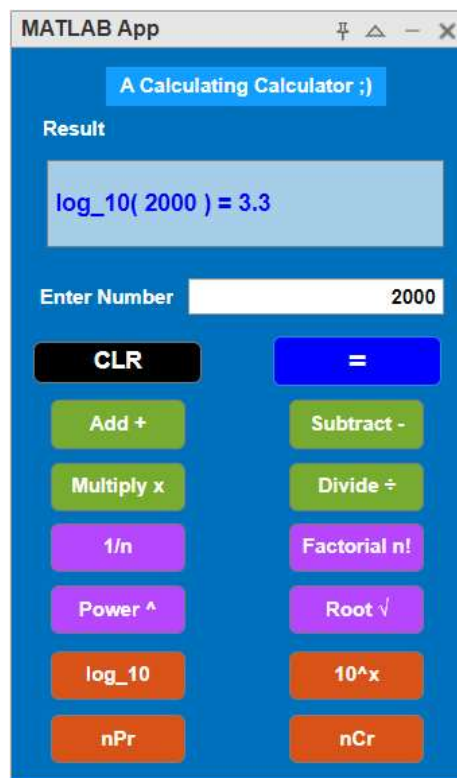


Fig 4.5 log Base 10 function

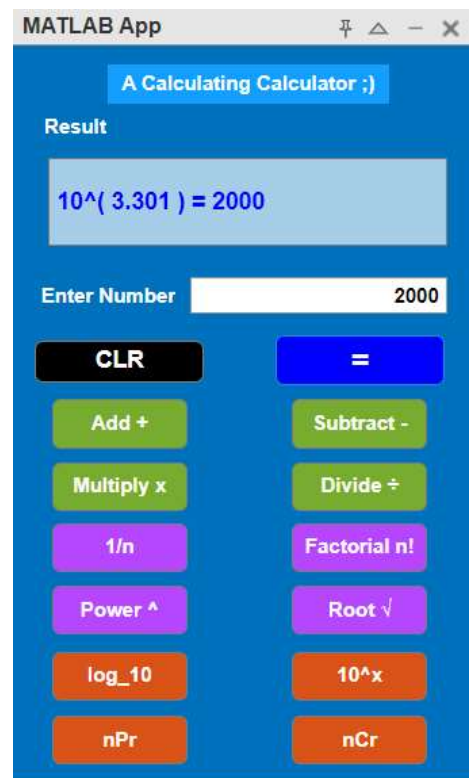


Fig 4.6 antilog (10^x) function

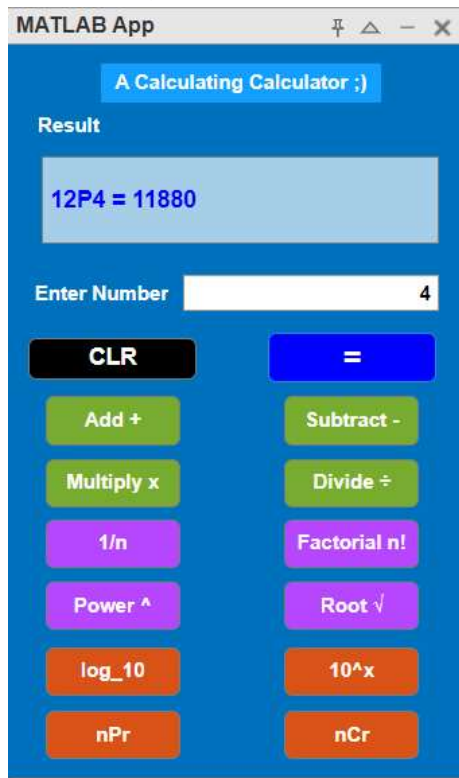


Fig 4.7 Permutation function

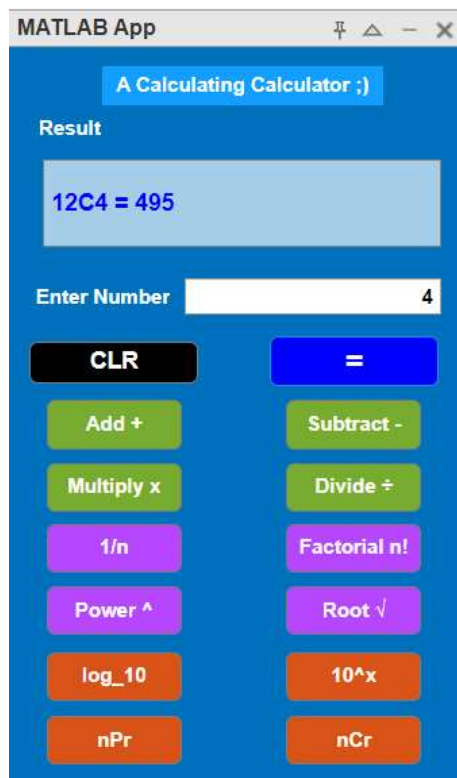


Fig 4.8 Combination function



Fig 5. Input validation

CONCLUSION

In the humble journey of crafting "A Calculating Calculator ;)," the project emerged as a labor of dedication and a commitment to providing a reliable mathematical tool. The intricate coding and failproof input mechanisms are a testament to the collective effort invested in creating a versatile and user-friendly calculator. Its feature-rich design, encompassing a range of mathematical operations, aims to cater to diverse user needs with a compact yet functional interface.

I extend my heartfelt gratitude to Prof. Bavrabi Ghosh for her choosing such a unique topic to build an application on, providing support, and guidance throughout the project's development. I am also grateful to the entire IT Workshop and MATLAB Lab for their invaluable contributions and assistance.